Operating Streaming Pipelines: Update, Drain, Diagnose

Updating a running job, the compatibility check, drain or cancel, snapshots, replaying from Pub/Sub, the three streaming signals, and finding a stuck step.

SECTION 5 of 5 EXAM WEIGHT 18% MODULE 37 of 45

admin@sthithapragna.com

A streaming job carries state. Every change is a question: what happens to the data in flight?

admin@sthithapragna.com

Four ways to change a running job

ACTION	IN-FLIGHT DATA	USE WHEN
Update with --update	Kept, with state	New code, compatible graph
Drain	Finished, windows closed early	Stopping, or an incompatible change
Cancel	Buffered data may be lost	Speed matters more than data
Snapshot, then new job	Saved, restored into the new job	Rollback, or a safe restart

THE TIEBREAKER

Losing nothing and keeping state is an update. Losing nothing but restarting is a drain.

Replacing a running job

- Launch with **--update**, the **same job name** and the same region.
- A **compatibility check** runs first. If it fails, the old job keeps running.
- State and in-flight records **carry over** to the replacement.
- Just changing the worker range? An **in-flight update**, with no new job.

> REMEMBER
>
> Batch jobs cannot be updated. Fix the code and run the job again.

What breaks the compatibility check

- **Renamed or removed** transforms, unless mapped with --transformNameMapping.
- **Adding or removing side inputs**, or changing a **coder**.
- Removing **stateful** steps, or moving the job to another **zone**.
- Small window changes are allowed. Schemas may **add** fields, never rename them.

> CURRENCY CHECK
>
> Test first: the graph_validate_only option checks compatibility without touching the running job.

Updating the readings pipeline

```
1 gcloud dataflow flex-template run meter-readings-prod \
2   --template-file-gcs-location=gs://lk-artifacts-prod/templates/readings-v42.json \
3   --region=europe-west2 --update \
4   --transform-name-mappings=ParseReading=ParseAndValidate
5
6 # Only the worker range changes? No new job needed:
7 gcloud dataflow jobs update-options JOB_ID --region=europe-west2 \
8   --min-num-workers=5 --max-num-workers=40
```

1-3 Same job name and region, with --update.

4 The step was renamed, so map old to new.

7 An in-flight update, for the evening peak.

HONEST LIMIT

If the check fails, the old job is untouched. Read the error, fix the graph, try again.

Drain or cancel?

DRAIN

Finish, then stop

- Stops reading new data
- Processes what is buffered
- Closes windows early, fires triggers
- Can take a long time

CANCEL

Stop now

- Halts at once
- Buffered data may be lost
- Releases resources quickly
- Cannot be restarted, only cloned

THE TRAP

Draining does not fix a stuck pipeline. A stuck job needs a code fix by update, or a cancel.

Saving state before a risky change

- A **Dataflow snapshot** saves a streaming job's state, optionally with its Pub/Sub sources.
- Start a new job from it with **--createFromSnapshot**, on Streaming Engine, same region.
- Kept for **7 days** by default, up to 30. Pub/Sub snapshots last at most 7 days.
- Sinks are not snapshotted. Writes since the snapshot may be written again.

> THE PATTERN
>
> Snapshot before every risky update. It is the rollback plan.

Replaying from Pub/Sub

- **Seek to a time**: messages after it become unacknowledged again, and are redelivered.
- Needs **retained acknowledged messages** on the subscription, or topic retention.
- **Seek to a snapshot**: return to a saved point, on any subscription to the topic.
- Replays deliver messages **again**. Downstream must deduplicate or overwrite.

> FOR LARKSPUR
>
> Take a Pub/Sub snapshot before deploying new subscriber code. Seek back if it misbehaves.

Three signals of a streaming job

SIGNAL	MEASURES	RISING MEANS
System lag	How long items wait or process, at most	A step is slow or stuck
Data freshness	How far output trails event time	Results are going stale
Backlog	Unread input, in bytes or seconds	Workers cannot keep up

THE TIEBREAKER

Backlog with low CPU: a stuck step or a hot key. Backlog with high CPU: too few workers.

Finding the stuck step

- Streaming retries a failing bundle **forever**, so one bad record can stall the job.
- Batch retries a bundle **four times**, then fails the whole job.
- '**Processing stuck** in step' means slow user code or an external call.
- Catch exceptions in the DoFn, and send bad records to a **dead-letter** output.

> REMEMBER
>
> The Diagnostics tab and worker logs, filtered to errors, point to the failing step.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
New code, **keep state**, no data loss	Update with --update
Update failed the **compatibility check**	Drain, then launch the new job
A step was **renamed**	--transformNameMapping
Roll back a bad deployment	Dataflow snapshot, or Pub/Sub seek
Job **stalls** on one bad record	Catch it, dead-letter output
Change only the **worker range**	In-flight update-options

THE PATTERN

Keep the state, finish the data, or stop now: decide before you touch the job.

> The readings pipeline holds an hour of deduplication state. New code renames one step and adds a field to the output.
>
> The team must deploy it without losing state or dropping readings.

How should they deploy?

A Cancel the job, then start the new code as a fresh job

B Update with --update and --transformNameMapping for the step

C Drain the job, then start the new code as a fresh job

D Run the new code beside the old job, writing to the same table

◆ PAUSE AND DECIDE

How should they deploy?

A Cancel and restart

Buffered data and all state are lost.

B Update with mapping

State carries over. The mapping passes the check.

C Drain and restart

No data lost, but the deduplication state is.

D Run both

Every reading written twice.

THE SIGNAL

New code, keep state, a renamed step: update with a name mapping.

> An update to the readings pipeline fails the compatibility check, because the new code removes a side input.
>
> The old job is still running. The new code must go live with no readings lost.

What should the team do?

A Force cancel the old job, then rerun the update command

B Cancel the old job, then launch the new code immediately

C Retry the same update with more workers, so the check passes

D Drain the old job, then launch the new code as a new job

◆ PAUSE AND DECIDE

What should the team do?

A Force cancel

Loses data, may leak VMs, and the check still fails.

B Cancel

Buffered readings are lost.

C More workers

Compatibility has nothing to do with workers.

D Drain, then launch

Buffered readings finish, then the new job starts.

THE SIGNAL

Incompatible change, no data lost: drain, then start the new job.

> The readings pipeline's backlog has grown for an hour and system lag keeps rising, but worker CPU is low.
>
> Worker logs repeat 'Processing stuck in step EnrichTariff' for several minutes at a time.

What is the most likely cause?

A. A slow or hanging external call inside the EnrichTariff step

B. Too few workers for the evening peak, so the job needs more of them

C. Pub/Sub has run out of retention, and is deleting messages

D. BigQuery streaming quota at the sink has been exceeded

◆ PAUSE AND DECIDE

What is the most likely cause?

A Slow external call

Low CPU, stuck in one step: waiting on something.

B Too few workers

High CPU would show that, not low.

C Retention

Messages are waiting, not being deleted.

D Sink quota

Errors would name the write step, not enrichment.

THE SIGNAL

Backlog with low CPU, stuck in one step: slow user code or an external call.

> A deployment of new code to the meter-alerts subscriber mis-flagged every reading for two hours, and acknowledged them all.
>
> The subscription retains acknowledged messages. The team has fixed the code, and needs those two hours reprocessed.

What should the team do?

A Create a new subscription, which receives every older message

B Ask the head-end to republish two hours of readings

C Seek the subscription to the time of the bad deployment

D Drain the subscriber, so the acknowledged messages are kept

◆ PAUSE AND DECIDE

What should the team do?

A New subscription

Only receives messages published after it exists.

B Republish

Slow, and the source may not hold them.

C Seek to the time

Messages since then become unacknowledged again.

D Drain

Finishes buffered work. It does not undo acks.

THE SIGNAL

Reprocess acknowledged messages: seek the subscription back in time.

Module 37 in five lines

- **Update** keeps state. Same name, --update, and a mapping for renamed steps.
- Check fails? **Drain**, then start the new job. Cancel may lose data.
- **Snapshot** before risky changes. Pub/Sub **seek** replays acknowledged messages.
- Lag, freshness, backlog: low CPU means **stuck**, high CPU means **too few workers**.
- Streaming retries **forever**. Dead-letter bad records, never let them stall.

COMING UP

Next: module 38, when pipelines fail: restarts, bad data and reprocessing.